

PG4200DataStructuresandAlgorithmsReport

This report presents our implementation, evaluation, and comparison of four fundamental sorting algorithms: Bubble Sort, Insertion Sort, Merge Sort and Quick Sort applied to the unique alcohol content values from the red and white Wine Quality Datasets. The alcohol values from datasets were combined and filtered using a `LinkedHashSet`. This ensured that all duplicates were removed while preserving the first occurrence order from the original datasets.

The aim of this report is to explain the logic behind each algorithm, show relevant code snippets, and analyze the performance of each sorting method using comparison counts and execution time. All algorithms' outputs were validated with Java's `Collection`. Sort to ensure correctness.

Before sorting, both CSV files were loaded using a custom *DataLoader* class. The alcohol values from the red and white wine datasets were extracted, combined, and filtered to ensure that only unique values were kept:

```
DataLoader loader = new DataLoader();

List<Double> redWine = loader.loadData( filePath: "src/main/resources/winequality-red.csv");
List<Double> whiteWine = loader.loadData( filePath: "src/main/resources/winequality-white.csv");

Set<Double> uniqueValues = new LinkedHashSet<>();
uniqueValues.addAll(redWine);
uniqueValues.addAll(whiteWine);
List<Double> allWines = new ArrayList<>(uniqueValues);
```

We used a `LinkedHashSet` to remove duplicate alcohol values, while converting it back to an array list. This provided a structure compatible with all sorting algorithms.

To evaluate how the algorithms behaved with a randomized list, a shuffled version was created using a fixed random seed to ensure consistent results between runs:

```
List<Double> shuffled = new ArrayList<>(allWines);
Collections.shuffle(shuffled, new Random( seed: 42L));
```

This allowed us to compare each algorithm's performance on both ordered and randomized input, revealing differences in comparison counts, recursion depth, and runtime behavior.

Bubble Sort

Bubble Sort works by repeatedly scanning through the full list and comparing pairs of values. If two values are in the wrong order, the algorithm swaps their places. Each complete pass of the outer loop moves the larger values toward the end of the list. Creating the sorted list gradually (Kartik, 2025).

```
int n = list.size();
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        comparisons++;

        if (list.get(j) > list.get(j + 1)) {
            double temp = list.get(j);
            list.set(j, list.get(j + 1));
            list.set(j + 1, temp);
        }
    }
}
```

For the unoptimized version the algorithm performs the full set of comparisons for every pass, even when parts of the list are already sorted. Because it does not check whether the list is partially sorted, the unoptimized version always executes the most possible number of comparisons. This results in consistent worst-case behavior of $\frac{n(n-1)}{2}$ which is inefficient.

The optimized version improves the unoptimized bubble sort by introducing a check during each pass that tracks whether any swaps have occurred during a pass. The Boolean flag is initially set to false and becomes true when a swap occurs. If a full pass completes without any swaps, the algorithm concludes that the list is already sorted and exits early (Kartik, 2025).

```
for (int i = 0; i < n - 1; i++) {
    boolean swapped = false;
    for (int j = 0; j < n - i - 1; j++) {
        comparisons++;

        if (list.get(j) > list.get(j + 1)) {
            double temp = list.get(j);
            list.set(j, list.get(j + 1));
            list.set(j + 1, temp);
            swapped = true;
        }
    }
    if (!swapped) { break; }
}
```

Quick Sort

Instead of scanning through the entire list repeatedly like Bubble Sort, quick sort works by selecting a pivot point and splitting the list into two groups consisting of values greater than the pivot value, and another group of values smaller than the pivot value. After partitioning, the algorithm recursively sorts the left and right sides. This makes Quick Sort more efficient because the size of the problem gets repeatedly reduced (Cormen et al., 2009).

In this project, Quick Sort was implemented using first, last, and a random value as the pivot points:

1. First value as pivot point
 - a. This is a simple strategy that chooses the first value in the list. But this strategy can perform poorly if the list is partially sorted or nearly sorted, because the partitions will in that case be consistently unbalanced.
2. Last value as pivot point
 - a. This works the same way as “first pivot” but using the last value instead and has the same weakness.
3. Random value as pivot point
 - a. This works by choosing the pivot point randomly between the low and high boundaries. This avoids predictable bad cases and has a higher probability of creating more balanced partitions, making it the most efficient strategy on average. (Knuth, 1998; Sedgewick & Wayne, 2011).

Below is the core Quick Sort implementation showing the recursive structure, pivot selection strategies and partitioning logic used in this project:

```

private void quickSort(List<Double> list, int low, int high, Pivot pivot) {
    if (low < high) {
        int pivotIndex = partition(list, low, high, pivot);
        quickSort(list, low, high: pivotIndex - 1, pivot);
        quickSort(list, low: pivotIndex + 1, high, pivot);
    }
}

private int partition(List<Double> list, int low, int high, Pivot pivot) {
    int pivotIndex = switch (pivot) {
        case FIRST -> low;
        case LAST -> high;
        case RANDOM -> low + random.nextInt( bound: high - low + 1);
    };

    double pivotValue = list.get(pivotIndex);
    Collections.swap(list, pivotIndex, high);

    int i = low;
    for (int j = low; j < high; j++) {
        comparisons++;
        if (list.get(j) <= pivotValue) {
            Collections.swap(list, i, j);
            i++;
        }
    }
    Collections.swap(list, i, high);
    return i;
}

```

Insertion Sort

Insertion Sort is a comparison-based sorting algorithm that builds a sorted list of one element at a time. It works in a way that is similar to how humans sort playing cards: values to the left are treated as the “sorted” portion, and the algorithm repeatedly takes the next unsorted element and inserts it into its correct position by shifting larger values to the right (Cormen et al., 2009; Sedgewick & Wayne, 2011).

During the implementation of insertion sort, the standard insertion sort algorithm was followed and included counters from comparisons and shifts. The operation started when alcohol values were extracted from the datasets “red wine” and “white wine” and stored in “LinkedHashSet”, this process was performed to ensure that only unique values were kept, and the original appearance order was preserved. Afterwards the core logic of algorithm was performed, the core logic is based on comparison where an element is compared to a key and shifting where an element is moved one position to the right. Code snippet below visualizes the core logic:

```
while (j >= 0) {  
    comparisons++;  
    if (a.get(j) > key) {  
        a.set(j + 1, a.get(j));  
        shifts++;  
        j--;  
    } else {  
        break;  
    }  
}  
a.set(j + 1, key);
```

The insertion sort was run twice once in original order of the unique alcohol values and once shuffled on a fixed random seed. These results were according to how the theory predicts, which predicts shuffled datasets to produce higher number of comparisons and shifts due to “values no longer being in a near-sorted structure”. (Knuth, 1998).

Merge Sort

Merge Sort is an efficient divide-and-conquer sorting algorithm that consistently achieves $O(n \log n)$ time in all best-, average-, and worst-case scenarios. First introduced by John von Neumann in 1945, merge sort works by repeatedly dividing the input list into halves until each sub-list contains only one element and then merging these sub lists back together in correct order (Knuth, 1998; Goodrich, Tamassia & Goldwasser, 2014).

In this operation, canonical recursive definition of merge sort was implemented while also tracking two performance metrics: merge operations (each time two sub lists are merged) and merge comparisons (each time two values are compared during merging). Below is the code snippet which demonstrates and visualizes how these two metrics were performed and incremented appropriately:

```
private List<Double> merge(List<Double> left, List<Double> right) {
    mergeOps++;
    List<Double> merged = new ArrayList<> (initialCapacity: left.size() + right.size());
    int i = 0, j = 0;
    while (i < left.size() && j < right.size()) {
        mergeComparisons++;
        if (left.get(i) <= right.get(j)) {
            merged.add(left.get(i++));
        } else {
            merged.add(right.get(j++));
        }
    }
}
```

Similar to insertion sort, merge sort was also run twice and the results also aligned with the merge sort theory which says: “The number of operations remain identical across both runs”. The reason behind this theory is that “merge operations depend solely on the size of dataset-and for a list of n values, merge sort performs exactly $n - 1$ merge, a fact supported by algorithm theory” (Cormen et al., 2009).

Analysis and Discussion

After combining the red and white wine datasets, we were left with 111 unique alcohol values that were extracted and stored using a `LinkedHashSet` to preserve the first-occurrence order. Each sorting algorithm was then evaluated on two inputs: the original ordered dataset and a shuffled dataset generated using a fixed random seed. For every run, we recorded comparison counts and execution times and verified correctness by comparing the output with `Collections.sort()`.

Bubble Sort (Unoptimized vs. Optimized)

The unoptimized Bubble Sort behaved as expected. Because it performs the full nested loop structure without checking if the list is already sorted, it generated 6105 comparisons on both the original and shuffled datasets. This matches the theoretical $\frac{n(n-1)}{2}$ behavior, with execution times 1.025 ms (original list), and 0.985 ms (shuffled list). With a slight difference due to system noise (Cormen et al, 2009; knuth, 1998).

The optimized version implements a Boolean flag to detect whether a pass resulted in any swaps. On nearly sorted input, an early exit reduces unnecessary work. However, since the original dataset of 111 unique alcohol values was not strongly ordered due to mixed red/white sequences. The optimized version still performed 5895 comparisons with the original list and 5934 comparisons with the shuffled list, only slightly lower than the unoptimized counts. This confirms that Bubble Sort is sensitive to input structure, and its best-case advantages appear only when the list is already sorted or nearly sorted.

Insertion Sort

Insertion Sort showed more adaptive behavior. On the original dataset, it required 2711 comparisons, increasing to 2977 comparisons with the shuffled list. This matches theoretical expectations: Insertion Sort performs efficiently when values are already close to their final positions, and performs progressively worse as disorder increases. Execution times 0.331 ms and 0.382 ms, also reflected this difference.

These results align with literature stating that Insertion Sort performs well on small, partially ordered datasets (Cormen et al., 2009; Knuth, 1998).

Merge Sort

Merge Sort produced the most stable performance across both runs because its behavior does not depend on input order. For a list of 111 values, it consistently performed 110 merge operations, as theory predicts Merge Sort always performs exactly $n-1$ merges. Comparisons inside merge operations were 600 with the original list and 620 with the shuffled list, a minor difference resulting from different left/right subarray layouts during merging. Execution times 0.339 ms and 0.435 ms remained low and consistent. These results confirm Merge Sort's theoretical time complexity of $O(n \log n)$ in all best, average, and worst-case scenarios. (Knuth, 1998; Goodrich et al., 2014)

Quick Sort - Pivot Strategy Comparison

Quick Sort was tested with three pivot strategies: first element, last element, and a randomly generated pivot.

Original dataset

- First pivot: 740 comparisons, 1.397 ms
- Last pivot: 685 comparisons, 0.149 ms
- Random pivot: 739 comparisons, 0.127 ms

Shuffled dataset

- First pivot: 751 comparisons, 0.113 ms
- Last pivot: 704 comparisons, 0.1 ms
- Random pivot: 739 comparisons, 0.127 ms

Across all runs, the last pivot and random pivot strategies performed most efficiently. The random pivot reduces the probability of consistently unbalanced partitions, while

the first pivot method is more vulnerable to worst-case structures. However, because the dataset is small, the performance differences appear primarily in comparison counts rather than runtime.

These results validate Quick Sort theory: pivot selection heavily influences recursion depth, and therefore the number of comparisons required (Cormen et al., 2009; Sedgewick & Wayne, 2011). Random pivoting is generally the safest average-case strategy.

Summary and Interpretation

The results of the algorithms closely reflected theoretical expectations. The unoptimized Bubble Sort produced the highest comparison count with 6105, while Merge Sort and Quick Sort achieved the most efficient performance due to their $O(n \log n)$ behaviour. Insertion Sort and the optimized Bubble Sort performed reasonably well because the dataset's structure provided partial ordering, though not enough to fully exploit best-case behavior.

Merge Sort demonstrated the greatest stability, performing nearly identical operations regardless of whether the dataset was ordered or shuffled. Quick Sort's performance depended heavily on pivot strategy, with the last-element and random-element pivots producing more balanced partitions and therefore fewer comparisons than the first-element pivot.

Although execution time differences were small due to the modest dataset size of 111 unique values, the comparison counts clearly illustrated the underlying algorithmic complexities. Overall, every algorithm produced correct results when validated against `Collections.sort()`, and the experiment outcomes aligned with established algorithm theory and the expectations outlined in the exam specification

Sources:

Kartik. (2025, July 23). *Bubble sort algorithm*. GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm>

Cormen, T. H., Lieserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

<https://www.cs.mcgill.ca/~akroitt/math/compsci/Cormen%20Introduction%20to%20Algorithms.pdf>

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison Wesley.

<https://algs4.cs.princeton.edu/home/>

Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting & Searching* (2nd ed.). Addison Wesley. <https://vivekupadhyay125.wordpress.com/wp-content/uploads/2013/08/donald-e-knuth-the-art-of-computer-programming-vol-3.pdf>

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java* (6th ed.). Wiley.

<https://dl.ebooksworld.ir/motoman/Wiley.Data.Structures.and.Algorithms.in.Java.6th.Edition.www.EBooksWorld.ir.pdf>